The undischarged-assumption relation OpenAssum is primitive recursive

The Open Logic Project contributors

openlogic:incompleteness/arithmetization-syntax:local-snapshot-20261013; CC BY 4.0

Editorial note. The proof's own definitions of Subseq and Subderiv are retained verbatim inside the counted span, so the bounded packing is self-contained given the coding conventions and SubtreeSeq.

Editorial note. The SubtreeSeq enumeration is cited from the computability-recursive-functions unit (proposition subtreeseq), where its primitiveness is proved; it is declared here as a navigational reference.

Editorial scope. One original proposition of the unit's natural-deduction-arithmetic section, counted once: primitiveness of the undischarged-assumption relation OpenAssum. The counted proposition and its author proof are retained in full; the preceding deriv proposition with its pointer proof and the following Prf proposition are separate results outside this card.

Allowed standard prerequisites (editorial).

- Natural-deduction derivation coding: the derivation with Goedel number d is a tuple whose first entry (d)_0 is the number of immediate subderivations, whose remaining entries are the immediate subderivations, and whose last component records the last rule and its discharge label; the function DischargeLabel extracting that label is primitive recursive.
- The assumption codification: an assumption derivation is the triple (0, z, n) where z is the Goedel number of the assumed formula and n its discharge label.
- SubtreeSeq: the primitive recursive function enumerating the Goedel numbers of all subderivations of a derivation (proved in the computability-recursive-functions unit of the same source tree, proposition subtreeseq; of length bounded by the code).
- Bounded quantification, sequence indexing, and concatenation are primitive recursive.

Source reference. Original proposition subtreeseq: sources/olp-computability-recursive-functions/pinned-original-source.tex, source lines 2192–2192. The proof uses the primitive recursive function SubtreeSeq to enumerate all subderivations before packing the path test into a bounded definition. It is a navigation pointer to material the counted proof cites; the counted proof retains its own argument

in full. The proposition is proved in the computability-recursive-functions unit of the same source tree.

Proposition 0.1. *The relation* OpenAssum(z, d) *that holds if z is the Gödel number of undischarged assumption A of the derivation δ with Gödel number d, is primitive recursive.*

Proof. An occurrence of an assumption is discharged if it occurs with label n in a sub-derivation of δ that ends in a rule with discharge label n. So A is undischarged assumption of δ if at least one of its occurrences is not discharged in δ. We must be careful: δ may contain both discharged and undischarged occurrences of A.

Consider a sequence δ_0, ..., δ_k where $\delta_0 = \delta$, δ_k is the assumption DisAn (for some n), and δ_{i+1} is an immediate sub-derivation of δ_i. If such a sequence exists in which no δ_i ends in an inference with discharge label n, then A is undischarged assumption of δ.

The primitive recursive function SubtreeSeq(d) provides us with a sequence of Gödel numbers of all sub-derivations of δ. Any sequence of Gödel numbers of sub-derivations of δ is a subsequence of it. Being a subsequence of is a primitive recursive relation: Subseq(s, s') holds iff $\forall i < \operatorname{len} s \exists j < \operatorname{len} s' (s)_i = (s')_j$. Being an immediate sub-derivation is as well: Subderiv(d, d') iff $\exists j < (d')_0 d = (d')_j$. So we can define OpenAssum(z, d) by

$$\begin{aligned}\exists s < \text{SubtreeSeq}(d)(&\text{Subseq}(s, \text{SubtreeSeq}(d)) \wedge (s)_0 = d \wedge \\ &\exists n < d((s)_{\operatorname{len} s \dot{-} 1} = \langle 0, z, n\rangle \wedge \\ &\forall i < (\operatorname{len} s \dot{-} 1)(\text{Subderiv}((s)_{i+1}, (s)_i) \wedge \\ &\qquad \text{DischargeLabel}((s)_i) \neq n))).\end{aligned}$$

□